Bayesian Quadrature with Subspace methods for Riemannian Laplace approximation

Simon Eiriksson

February 2025

1 Introduction

This paper presents a method for uncertainty estimation in machine learning models, exploiting the curvature of the loss function and Bayesian Quadrature, to make efficient parameter samples from the posterior distribution.

First we present the Vanilla Laplace approximation that approximates the posterior distribution by a Gaussian distribution with the same mean and covariance. This approximation does not work well for high dimensional models, since the posterior distribution tends to be more non-gaussian with increasing dimensionality. The idea with the Riemannian Laplace approximation (Riemannian LA) is to exploit the knowledge we have of the loss surface. By introducing a Riemannian metric on the parameter space, we can "guide" the Laplace samples away from areas with low likelihood. However, the computation of Riemannian Laplace samples is highly computationally expensive.

This leads the contribution of this paper: The Riemannian LA induces a distribution over the parameter space that we want to sample from, in order to get samples from the *predictive* posterior. Assuming that the predictive posterior distribution is drawn from a Gaussian process, we can use samples from the Riemannian LA to retrieve data points as conditions to the Gaussian process. Having a conditional Gaussian process allows us to use Bayesian quadrature to estimate the mean of, and sample the predictive posterior distribution.

Furthermore, by sampling only from a subspace of the full parameter space, the Gaussian process can be restricted to a corresponding subspace. This allows us to integrate the predictive posterior Gaussian process using only a limited number of samples from the Riemannian Laplace approximation.

2 Bayesian Inference

In predictive machine learning, we train a model that makes a prediction y, given some input x. If the model is a neural network, the most common approach is to train a model that makes a point estimate for y. For many purposes, it is, however, useful to get an estimate of how sure the model is about the prediction. This for example can be important when there are few training observation, or if we want to know how well out-of-distribution (OOD) samples are predicted.

One approach to this is through Bayesian inference methods. Assume that the model is parameterized by $\theta \in \Theta = \mathbb{R}^d$ and that the data set is denoted $\mathcal{D} = \boldsymbol{t}|\boldsymbol{x}$. Then Bayes rule gives us the *posterior* distribution of the parameters, given the model architecture and the data that we have

already seen:

$$p(\theta|\mathcal{D})p(\mathcal{D}) = p(\mathcal{D}|\theta)p(\theta) \Rightarrow p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \tag{1}$$

Here the *prior* distribution of the parameters $p(\theta|\mathcal{D})$ is any knowledge we had about the parameter distribution before seeing our training data $\mathcal{D}$. The denominator $p(\mathcal{D})$ in the fraction is a normalizing constant, that ensures that the fraction sums to one, when integrated over θ. In general this value is untractable for practical machine learning tasks, as it is given by the integral

$$p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta,$$

which is not analytically solvable. For this reason, a range of approximative methods has been proposed, such as MCMC methods, Laplace approximation, Variational Inference and others. This paper discusses a combination of Laplace approximation, Riemannian Laplace approximation, and Bayesian Quadrature methods.

Assuming for a moment that the above hurdle is overcome, and that we want to estimate the predicted distribution of the target t^* given a new input x^*, we can then integrate over the parameter distribution to get the *posterior predictive*:

$$p(t^*|x^*) = \int p(t^*|x^*, \theta) \cdot p(\theta|\mathcal{D})d\theta \tag{2}$$

3 Examples

Throughout the paper, two examples will be used. The first is the *banana* distribution, which is a 2d Gaussian, which has been squeezed into a half-moon shape as depicted in figure 1a:

$$p(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\frac{1}{2}(x/\sigma_x)^2 + (y - c \cdot x^2)^2/\sigma_y^2\right)$$

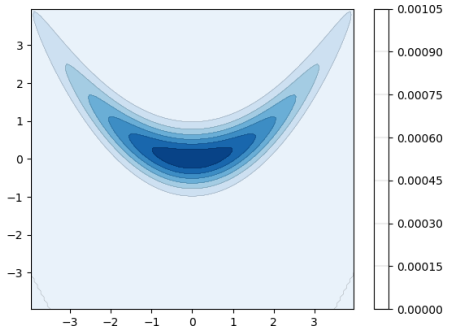
where c is a curvature constant that controls the squeeziness.

The second example is a regression problem using a small neural network as a function approximator. The model takes a one-dimensional input and output, has two hidden layers of each 10 neurons, and uses ReLU activation function. The network has been randomly initialized, 50 training data samples has been generated with $\sigma_t = 0.05$ sampling noise and finally, the model has been fine-tuned to fit to the sampled data (to guarantee that the parameters actually reflects a minimum in the loss landscape with respect to the training data). What we want to see here is a model that is certain about the mean value in the areas where there are data points, but expresses higher uncertainty in the interval $[-0.5, 0.5]$ where there is no training data available.

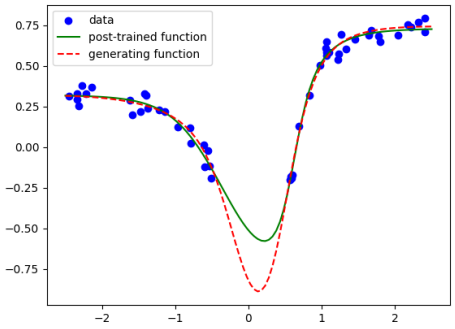
4 Laplace approximation

One approach to approximating the posterior parameter probability, is to use a second degree polynomial to approximate the aggregate loss function $\mathcal{L}(\theta) \propto -\ln p(\mathcal{D}|\theta) - \ln p(\theta)$. In practice, this is done by finding a Maximum Aposteriori (MAP) estimate θ^*, that minimizes $\mathcal{L}(\theta)$. The Laplace approximation is then the normal distribution with mean θ^* and the same covariance matrix Σ_0 as $p(\theta|\mathcal{D})$ has locally around θ^*.

Let B_0 be the hessian of the loss, with respect to the parameters: $B(\theta) \coloneqq -\mathbf{H}_\theta[\mathcal{L}(\theta)]$. Then if $\Sigma_{\theta^*} = B^{-1}(\theta^*)$ is positive semi-definite, we can define a Gaussian distribution $q(\theta) \sim \mathcal{N}(\theta^*, \Sigma_{\theta^*})$



(a) Banana probability density with $c = 0.2$, $\sigma_x = 2$, and $\sigma_y = .5$



(b) Function approximator

Figure 1: Examples used in this paper

which has the property that $\mathbf{H}_\theta[\ln q(\theta)]|_{\theta^*} = \Sigma_{\theta^*}$, meaning that the log of the approximate distribution $q(\theta)$ has the same Hessian as the log of the true posterior distribution $p(\theta|\mathcal{D})$.

For smaller models, and models where the posterior distribution is guaranteed locally approximately Gaussian, this can be a good solution. However, as the dimensionality of the parameter space increases, the probability that the posterior distribution is non-Gaussian along at least one of the axes increases, leading to low-likelihood parameter samples.

The Laplace approximation class used in this paper is built to allow for subspace sampling. That is, if A is an eigenvector decomposition of Σ_{θ^*} such that $A^\top A = \Sigma_{\theta^*}$ and $\varepsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_d)$ then we also have that $\theta = \theta^* + A\varepsilon$. If we then let A_k be the first k columns of A, and sample $\varepsilon_k \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_k)$, we can reduce the space that θ is sampled from, by setting $\theta_k = \theta^* + A\varepsilon_k$.

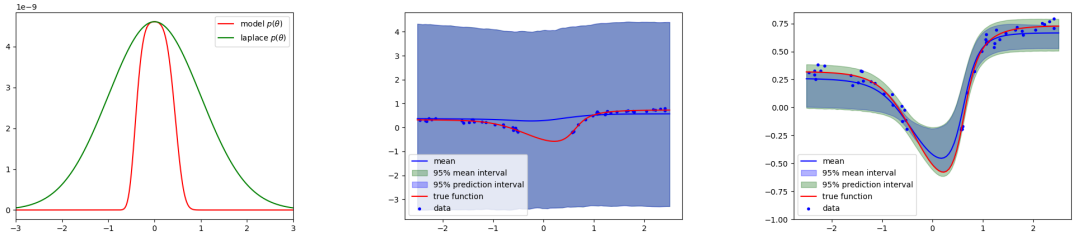
In figure 2a, the posterior density of the function approximator, sliced along the direction of the eigenvectors belonging to the two dominant eigenvalues of the covariance matrix, is displayed along with its Laplace approximation. As can be seen in this example, the Laplace approximation has relatively much support for parameter values that gives rise to high loss. This is problematic, since parameter samples from these areas will result in wildly wrong prediction functions, which in turn means too wide prediction bands.

Similarly for the universal function estimator. When using the full Laplace approximation, we see some terrible parameter sampling with a mean that does not follow the true mean, and extremely wide prediction bands as seen in figure 2b. If samples are only taken from the most important eigendirections of the covariance matrix, the results improve drastically as seen in figure 2c. We do however still see much overestimation of uncertainty due to highly improbable parameter samples as seen in figure 2a.

5 Riemannian Laplace Approximation

In comes the Riemanninan Laplace approximation. By using samples from the Laplace approximation and taking into account the curvature of the loss landscape parameters which are far away from our MAP solution in terms of the the curvature of the loss landscape get penalized.

We define the loss manifold as $\mathcal{M} = g(\theta) = [\theta, \mathcal{L}(\theta)]$ residing in $\mathbb{R}^{d+1}$ and consider a curve



(a) First eigendirection of the function approximator posterior

(b) Full rank Laplace approximation of the function approximator

(c) Rank-2 Laplace approximation of the function approximator

Figure 2: Laplace approximations

$c : [0, 1] \mapsto \Theta$ on the parameter space parameterized by $t \in [0, 1]$ [1]. If then $\dot{c}(t)$ is the time derivative of the curve, and $\boldsymbol{J}_{c(t)} = \left.\frac{\partial g}{\partial \theta}\right|_{\theta=c(t)}$ is the Jacobian of g taken in the point $\theta = c(t)$, following [Hau25] we can write out the length of the curve as

$$\begin{aligned}\text{Length}_{\mathcal{M}}(c) &\coloneqq \int_0^1 \left\| \frac{d}{dt} g(c(t)) \right\| dt \\ &= \int_0^1 \left\| \boldsymbol{J}_{c(t)} \dot{c}(t) \right\| dt \\ &= \int_0^1 \sqrt{\dot{c}(t)^\top \boldsymbol{J}_{c(t)}^\top \boldsymbol{J}_{c(t)} \dot{c}(t)} dt.\end{aligned}$$

Following [Ber+23] we can use that $\boldsymbol{J}_{c(t)} = [\mathbb{I}_d, \nabla_\theta \mathcal{L}(\theta)]$ to define the *Riemannian matric* on $\mathcal{M}$ as

$$M(\theta) = \boldsymbol{J}_{c(t)}^\top \boldsymbol{J}_{c(t)} = \mathbb{I}_d + \nabla_\theta \mathcal{L}(\theta) \nabla_\theta \mathcal{L}(\theta)^\top \tag{3}$$

This notion of distance induces two definitions.

First the geodesic, which is the which is a shortest curve between two points (θ_0, θ_1) on the manifold:

$$c^* = \operatorname*{argmin}_c \text{Length}_{\mathcal{M}}(c) \tag{4}$$

$$\text{subject to } c(0) = \theta_0 \text{ and } c(1) = \theta_1 \tag{5}$$

Finding the geodesic between two points corresponds to solving a boundary value problem for a differential equation and is generally computationally expensive. The directional vector at time 0, given by $\dot{c}^*(0)$ uniquely determines the curve (as we shall shortly see). This motivates the definition of the Log-map, which takes two points (θ_0, θ_1) in parameters space as input and returns a vector in parameter space[2]:

$$\text{Log}_{\theta_0}(\theta_1) \coloneqq \dot{c}^*(0)$$

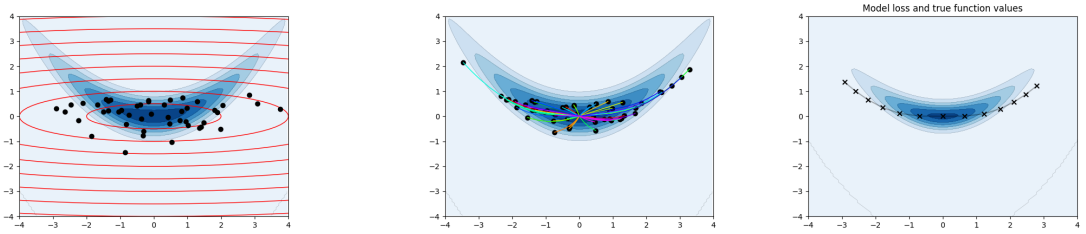
[1] In the literature the curve is on the manifold. The function $g(\theta)$ gives a one-to-one correspondence between Θ and $\mathcal{M}$, so we can think of any operation taking place on either of the two, as we like.

[2] Corresponding to the previous note: the literature defines log Log-map on the manifold and returns a directional vector in $\Theta \times \mathbb{R}$

The Log-map is thus the initial velocity of the geodesic that starts from θ_0 and reaches θ_1 in one time unit [Frö+21]. The "inverse" of this mapping is the Exp-map that given a starting point θ^* and a directional vector v returns the point that is reached by moving in direction v from θ^* on the loss manifold for one time unit. That is, if $\text{Exp}_{\theta_0}(v) = c(1)$ then $v = \text{Log}_{\theta_0}(c(1))$. The Exp-map corresponds to solving the initial value problem for the differential equation [Ber+23]:

$$\ddot{c}(t) = -\nabla_\theta \mathcal{L}(c(t)) \left(1 + \nabla_\theta \mathcal{L}(c(t))^{-1} \nabla_\theta \mathcal{L}(c(t))\right)^{-1} \langle \dot{c}(t), \mathbf{H}_\theta[\mathcal{L}](c(t))\dot{c}(t)\rangle. \tag{6}$$

It then holds that $\text{Exp}_{\theta_0}(\text{Log}_{\theta_0}(v)) = v$. Note however that the Log-map is not uniquely defined, so the reverse does not necessarily hold. Furthermore, it holds that if c^* is the geodesic between θ_0 and θ_1, and $v = \text{Log}_{\theta_0}(\theta_1)$, then for any $t \in [0, 1]$ it also holds that $c^*(t \cdot v) = \text{Exp}_{\theta_0}(t \cdot v)$.



(a) Laplace approximation to the banana distribution

(b) Riemannian Laplace samples from banana distribution

(c) Riemannian Laplace samples from rank-1 subspace of the 2d banana distribution

Figure 3: Riemannian Laplace approximation

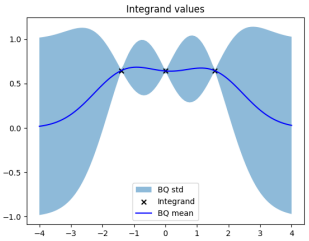
In figure 3a, 50 samples from the Laplace approximation together with corresponding red isocurves for the Laplace density are overlayed the Banana distribution. The Exp-map of the same samples are seen in figure 3b together with their corresponding geodesics. In figure 5a, we see the resulting function approximator with prediction bands, as given by 10 samples along the first eigendirection of the Riemmanian LA.

6 Bayesian Quadrature

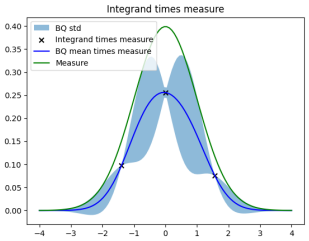
A major issue with the Riemannian LA is the computational cost of solving the IVP-problem. In the following, we will discuss how to remedy this by using Bayesian quadrature to efficiently find the integral in equation 2.

Bayesian quadrature is a numerical method for integrating a function $f(\theta)$ over some measure $\pi(\theta)$: $J = \int f(\theta)\pi(\theta)d\theta$. Briefly speaking, the assumption is that the function we want to integrate is drawn from a Gaussian Process distribution. Then the integral J is a random variable and given a set of observations of θ and $f(\theta)$, the mean and variance of J can be found. If the integration measure π furthermore is Gaussian, the mean and variance has an analytical solution [Gun+14].

The amount of expected variance reduction gain from a new datapoint is measured by the acquisition function, and by maximizing this function it is computationally feasible to find the next value of θ that will minimize the variance of J. Figure 4a shows then mean and variance of a Gaussian Process conditioned on three data points as an illustration. The corresponding function values and variance multiplied by the integration measure is seen in figure 4b.



(a) Integrand values



(b) Integrand values and measure

Figure 4: Bayesian Quadrature approximation

In our case, $f(\theta) = p(t^*|x^*, \theta)$ and $\pi(\theta) = p(\theta|\mathcal{D})$. As discussed in the previous section we approximate $p(\theta|\mathcal{D})$ by the Riemannian LA denoted by $q_\theta(\theta)$ where $\theta = \text{Exp}_{\theta^*}(v)$ and $v \sim \mathcal{N}(\theta^*, \Sigma_{\theta^*})$. Then $v = \text{Log}_{\theta^*}(\theta)$ and if the distribution of v is given as $q_v(v)$, following [Tom24, p.29] we get that

$$q_\theta(\theta) = q_v(\text{Log}_{\theta^*}(\theta)) \cdot \left| \det \left(\frac{\partial \text{Log}_{\theta^*}(z)}{\partial z}\bigg|_{z=\theta} \right) \right| \tag{7}$$

$$= q_v(\text{Log}_{\theta^*}(\theta)) \cdot \left| \det \left(\frac{\partial \text{Exp}_{\theta^*}(z)}{\partial z}\bigg|_{z=\text{Log}_{\theta^*}(\theta)} \right) \right|^{-1} \tag{8}$$

$$= q_v(\text{Log}_{\theta^*}(\theta)) \cdot |J[\text{Exp}_{\theta^*}](\text{Log}_{\theta^*}(\theta))|^{-1} \tag{9}$$

$$\tag{10}$$

from which we can conclude that

$$q_v(v) = q_\theta(\text{Exp}_{\theta^*}(v)) \cdot |J[\text{Exp}_{\theta^*}](v)|$$

We can expand on the primary integral with integration by substitution and get

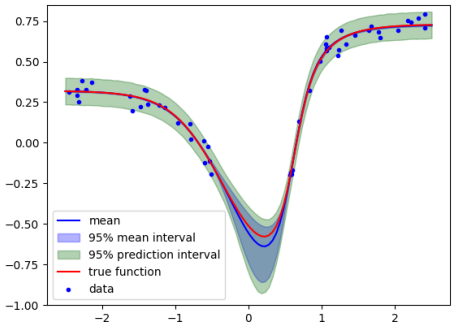
$$\begin{aligned} J &= \int f(\theta) q(\theta) d\theta \\ &= \int f(\text{Exp}_{\theta^*}(v)) \cdot q_\theta(\text{Exp}_{\theta^*}(v)) \cdot |J[\text{Exp}_{\theta^*}](v)| dv \\ &= \int f(\text{Exp}_{\theta^*}(v)) \cdot q_v(v) dv \end{aligned}$$

Since $q_v(v)$ is Gaussian, we can use this as measure for the Bayesian quadrature, and then integrate the function $f(\text{Exp}_{\theta^*}(v))$.

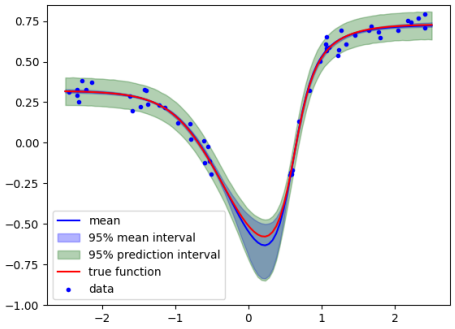
In figure 5b, the Bayesian Quadrature Riemannian Laplace approximation (BQ-Riemann) for the function approximator is plotted with the corresponding 95% band for the mean and for the observations.

7 Implementation

The Laplace approximation has been implemented in `Pytorch` [Ans+24] using Generalized Gauss-Newton approximation for convex loss functions to guarantee that Σ_{θ^*} is positive semidefinite, as



(a) Riemannian Laplace approximation for the function approximator



(b) Bayesian Quadrature Riemannian Laplace approximation for the function approximator

Figure 5: Function approximator

discussed in [DAM24; ksn25]. The example with the banana distribution has been implemented with a loss that is linear in the model parameters, and therefore uses the exact Hessian.

The Bayesian quadrature model is implemented with `emukit` [Pal+19; PML23], and uses `GPy` [GPy12] for the Gaussian process modeling. All GP models in this paper uses the `GPy` Gaussian process model for regression based on a Gaussian kernel with unit length-scale and variance.

The differential equations for the Riemannian metric are solved with `torchdiff` [Che21]. The BQ-Riemann experiments uses 7 time steps, while the Riemannian LA experiments uses 10 time steps.

Like [Frö+21], we exploit the fact that the solution of the IVP is done through discrete steps, where the function value (the model output) is evaluated at each step. Using `torchdiff`, we get evaluations at equidistant time steps $t \in \{k/n | k = 1 \dots n\}$ corresponding to parameters $\theta_k = \text{Exp}_{\theta^*}(v \cdot k/n)$. The acquisition function in the quadrature model is then extended to include the sum of the acquisition function evaluated at each point $v \cdot k/n$. This is useful in the Bayesian Quadrature model, where each model evaluation thus can be used in approximating the integral.

In figure 6, the result of 8 samples from the rank-2 Laplace approximation is plotted, with their corresponding traces. The intuition is that the model aims to approximate the BQ Model mean in the lower left panel

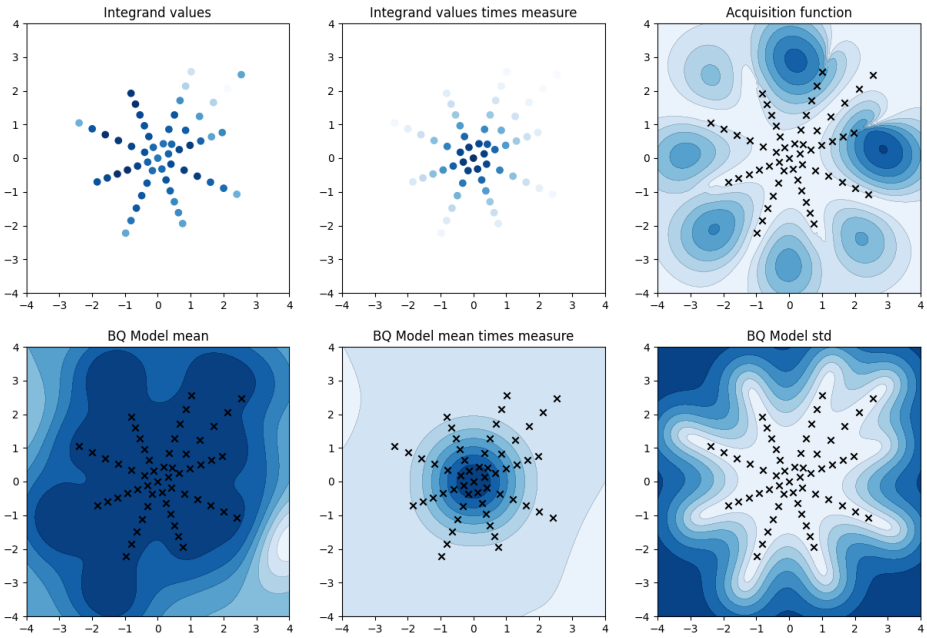


Figure 6: Bayesian Quadrature. All figures represents model evaluations in one data point, and are seen from the space of the v-vector. Clockwise from upper left: The integrand values are the model evaluations, given the rays of the v's. Here, 9 different values of v has been explored and their corresponding ray traces can be seen. The second panel show the model evaluations multiplied by the integration measure - which is a standard normal normal distribution. The acquisition function in the third panel returns the expected variance reduction in the integral of the Gaussian Process, given that we evaluate our model in the specific point. The fourth panel shows the expected value of the underlying GP, whereas the fifth shows the underlying GP multiplied by the integration measure. The last panel shows the variance of the expected value of the underlying GP

As noted before, the Vanilla Laplace model allows for subspace sampling. This property is inherited in the other models. The idea extends to the Riemannian LA where the samples will lie close to the sub-manifold of the original loss manifold, extended by the most important eigenvectors from the Laplace approximation. In the case with a 1-dimensional sub-manifold within a 2-d loss manifold, we see that the samples lie on the same path that tends to lie close to the ridge induced by the most important eigenvector. See figure 3c for an intuition of the idea.

8 Experiments

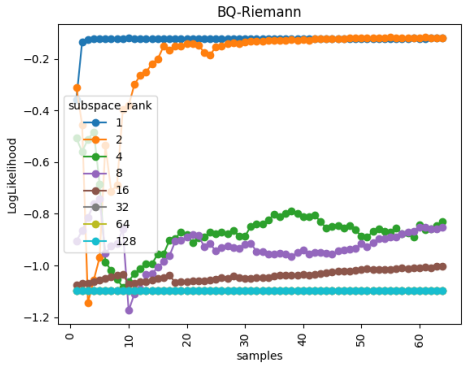
We test the Bayesian Quadrature Riemannian Laplace approximation using a small toy classification data set and the UCI Wine data set[Cor+09]. In both experiments, we compare the BQ-Riemann with the Riemannian LA, a vanilla Laplace approximation and the MAP solution. In the BQ-

Riemann model, the predictive distribution is approximated by Monte Carlo sampling where we evaluate the Gaussian process in 10.000 points and calculate mean and prediction intervals from this. The Riemannian LA model predictions are calculated as the mean of the model output, given each of the parameter samples. The vanilla Laplace model predictions are calculated using 10.000 samples from the Laplace parameter distribution.

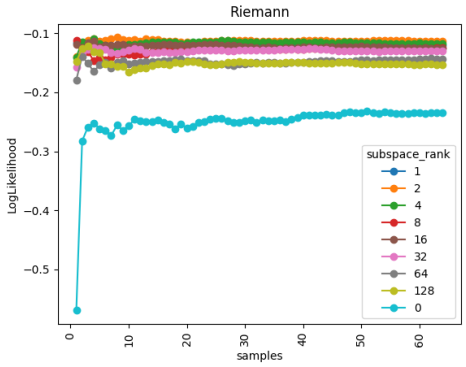
8.1 UCI wine classification

The model used for the UCI wine classification is an MLP that takes 13 inputs variables, has two hidden layers with each 26 neuron connected by a ReLU activation function, and a softmax activation function at the output.

The experiment was run with subspace ranks $[1, 2, 4, 8, 16, 32, 64, 128, \text{full rank}]$, and 64 samples were used for the Riemannian LA and the BQ-Riemann models. The resulting log-likelihood can be seen in table 1. For comparison, the MAP log-likelihood is -0.1330. Thus, it is clear that both the BQ-Riemann and the Riemannian LA approximation have slightly higher likelihood than the MAP estimate. However, it is also clear that the BQ-Riemann approximation deteriorate quickly as the subspace rank increases. This is probably due to the low number of samples used in the BQ-Riemann approximation, compared to the dimensionality of the sampling space. As can be seen in figure 7a, the log likelihood of the test data converges within two samples for the rank-1 approximation, and within around 20 samples for the rank-2 approximation. However the approximations with higher rank sampling space never reaches saturation, and for rank 32, 64, and 128, the models yields very low likelihood. The main takeaway is that *many* samples are necessary for higher dimension BQ-Riemann models. On the other hand, the 1- and 2-dimensional BQ-Riemann models yields reasonably fast and good quality predictions.



(a) BQ-Riemann: Log likelihood as function of number of samples in the UCI-wine experiment



(b) Riemannian-LA: Log likelihood as function of number of samples in the UCI-wine experiment

Figure 7: UCI wine classification experiment

Table 1: UCI Wine LogLikelihood

Subspace rank	BQ-Riemann	Laplace	Riemann
1	-0.1210	-0.1582	-0.1202
2	-0.1182	-0.1592	-0.1130
4	-0.8297	-0.1601	-0.1175
8	-0.8511	-0.1898	-0.1251
16	-1.0043	-0.2177	-0.1254
32	-1.0979	-0.2503	-0.1294
64	-1.0986	-0.3112	-0.1432
128	-1.0986	-0.4575	-0.1533
Full rank	NaN	-1.0592	-0.2349

8.2 Synthetic classification problem

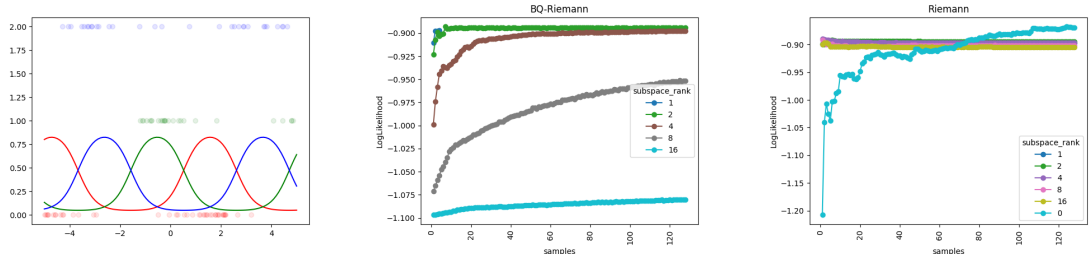
The toy dataset consists of a single input dimension, and three classes as seen in figure 8a, where the 100 training samples from the three classes are represented by dots, and the corresponding underlying true probabilities are represented by the three coloured lines. The classification model is an MLP with two hidden layers with 50 neurons each connected by a ReLU activation function, and a softmax activation function at the output.

The experiment was run with subspace ranks $[1, 2, 4, 8, 16, \text{full rank}]$, and 128 samples were used for the Riemannian LA and the BQ-Riemann models. The resulting log-likelihood can be seen in table 2. For comparison, the MAP log-likelihood is -0.8961, meaning that the log-likelihood is on par with the models with lower rank. As is the case with the UCI wine task, the the Laplace and BQ-Riemann models deteriorates when the rank increases.

We also see that the BQ-Riemann model needs *many* samples in order to converge when the rank increases. Curiously, the full rank Riemannian LA model seems to have increasing log-likelihood, as the sample number increases, oposed to the results from the UCI wine data set, where the full rank Riemannian LA yielded very bad results. In the experiment we only made 128 samples, since the compute cost is significant, but this result indicates that the full rank Riemannian Laplace might converge to an even better performance, whereas the BQ-Riemann models seem to stagnate.

Table 2: Synthetic data classification LogLikelihood

Subspace rank	BQ-Riemann	Laplace	Riemann
1	-0.8973	-0.9085	-0.8975
2	-0.8943	-0.8893	-0.8941
4	-0.8976	-0.8922	-0.8968
8	-0.9518	-0.9338	-0.9013
16	-1.0800	-1.0081	-0.9049
Full rank	NaN	-1.1074	-0.8695



(a) Synthetic data classification task: samples and probabilities

(b) BQ-Riemann: Log likelihood as function of number of samples in the UCI-wine experiment

(c) Riemannian-LA: Log likelihood as function of number of samples in the UCI-wine experiment

Figure 8: Synthetic data classification experiment

9 Conclusion

In this paper, we have presented and compared three different approaches to Bayesian uncertainty estimation: Vanilla Laplace approximation, Riemannian Laplace approximation (Riemannian LA), and Bayesian Quadrature Riemannian Laplace approximation (BQ-Riemannian).

It is clear from the experiments that the BQ-Riemann performs on par with the Riemannian LA, as long as the subspace rank of the models is at most 4. This is probably mainly due to the variance induced by the Bayesian Quadrature model with increasing subspace rank. In general, it is fair to conclude that the Bayesian quadrature approach does not scale to high dimensions.

In future work, it may be advised to try out different Gaussian process kernels such as the Matérn kernel as proposed in [Frö+21]. Furthermore it might be an option to tweak or otherwise optimize the Gaussian process parameters (length-scale and variance), to better suit higher-dimensional models.

10 Future work

- Run on a practical dataset such as FashionMNIST or some of the smaller UCI regression data sets.
- Implement with usage of Laplace Redux [Dax+22] to make use of diagonalized/Kronecker Laplace in IVP-integration.
- Implement this with Laplace linearization
- Implement Subspace Inference as [Izm+20], to reduce the effective number of model parameters.

References

[Cor+09] Paulo Cortez et al. *Wine Quality*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C56S3T 2009.

[GPy12] GPy. *GPy: A Gaussian process framework in python*. `http://github.com/SheffieldML/GPy`. 2012.

[Gun+14] Tom Gunter et al. "Sampling for inference in probabilistic models with fast Bayesian quadrature". In: *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*. NIPS'14. Montreal, Canada: MIT Press, 2014, pp. 2789–2797.

[Pal+19] Andrei Paleyes et al. "Emulation of physical processes with Emukit". In: *Second Workshop on Machine Learning and the Physical Sciences, NeurIPS*. 2019.

[Izm+20] Pavel Izmailov et al. "Subspace Inference for Bayesian Deep Learning". In: *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*. Ed. by Ryan P. Adams and Vibhav Gogate. Vol. 115. Proceedings of Machine Learning Research. PMLR, July 2020, pp. 1169–1179. URL: https://proceedings.mlr.press/v115/izmailov20a.html.

[Che21] Ricky T. Q. Chen. *torchdiffeq*. Version 0.2.2. June 2021. URL: https://github.com/rtqichen/torchdiffeq.

[Frö+21] Christian Fröhlich et al. *Bayesian Quadrature on Riemannian Data Manifolds*. 2021. arXiv: 2102.06645 [cs.LG]. URL: https://arxiv.org/abs/2102.06645.

[Dax+22] Erik Daxberger et al. *Laplace Redux – Effortless Bayesian Deep Learning*. 2022. arXiv: 2106.14806 [cs.LG]. URL: https://arxiv.org/abs/2106.14806.

[Ber+23] Federico Bergamin et al. "Riemannian Laplace approximations for Bayesian neural networks". In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: https://openreview.net/forum?id=bHS7qjLOAy.

[PML23] Andrei Paleyes, Maren Mahsereci, and Neil D. Lawrence. "Emukit: A Python toolkit for decision making under uncertainty". In: *Proceedings of the Python in Science Conference* (2023).

[Ans+24] Jason Ansel et al. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation". In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ASPLOS '24. La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 929–947. ISBN: 9798400703850. DOI: 10.1145/3620665.3640366. URL: https://doi.org/10.1145/3620665.3640366.

[DAM24] Yann N. Dauphin, Atish Agarwala, and Hossein Mobahi. *Neglected Hessian component explains mysteries in Sharpness regularization*. 2024. arXiv: 2401.10809 [cs.LG]. URL: https://arxiv.org/abs/2401.10809.

[Tom24] Jakub M Tomczak. *Deep Generative Modeling*. Springer Cham, 2024.

[Hau25] Søren Hauberg. *Differential geometry for generative modeling*. Feb. 2025. URL: https://www2.compute.dtu.dk/~sohau/weekendwithbernie/Differential_geometry_for_generative_modeling.pdf.

[ksn25] ksnxr. *Generalized Gauss-Newton (GGN)*. Accessed: 2025-01-05. 2025. URL: https://ksnxr.github.io/ggn.html.